

Beyond the PR: The New Control Plane for Agentic Software Delivery (Webinar Sept 17th).

Spec Driven Development: When Architecture Becomes Executable

Key Takeaways

- Spec-Driven Development (SDD) represents a fifth-generation programming shift, elevating abstraction to the system level. Engineers define intent declaratively, and platforms materialize execution through generation and validation.
- Architecture is no longer advisory; it is executable and enforceable. Specifications now define the system truth, while code is continuously generated, validated, and regenerated.
- Drift detection transforms architecture from a design-time artifact into a self-policing control system through continuous schema validation, contract testing, payload inspection, and spec diffs.
- Human authority shifts upward from implementation to intent, policy, and ethics. SDD does not remove humans from the loop; it relocates their responsibility to the highest abstraction layers.
- SDD provides architectural determinism, reduced systemic drift, and multi-language parity, but introduces new complexity surfaces and a significant cognitive shift for engineers.

A Fifth Generation of Abstraction

Every major shift in the history of software engineering has been driven by one consistent force: the rise of abstraction. The earliest generation of software was written in raw machine code before Assembly introduced a layer of readability and control. Higher-level languages have evolved across multiple distinct paradigms, enabling the development of general-purpose languages such as C, Java, and Python, and their derivatives.

These languages enabled advances in abstraction, in which concepts such as memory management and platform-specific quirks were masked from the day-to-day workflow and handled on the developer's behalf. This level of accessibility allowed for the growth of the broader ecosystem, as with each emergent

generation of languages, complementary supporting toolchains evolved.

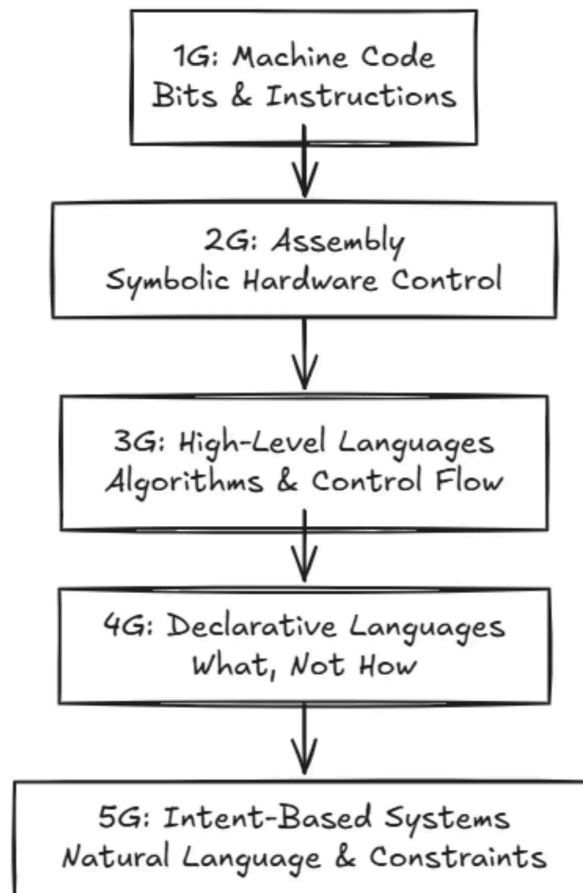


Figure 1: Generations of Programming Language

The fifth generation, that of using natural language, has long been the targeted evolution of programming languages wherein humans conversed in their native language, and it is interpreted in an executable manner. This evolution has not truly become mainstream until recently. Driving this is the generational capability of Artificial Intelligence (AI), which is now maturing to take that human-centric input and build solutions in a programming language of your choice. This evolutionary scene is documented in decades of academic research and discussed informally in blogs.

A common theme across these transitions is the evolving role of the developer. Each abstraction elevation allowed the developer to focus more on intent and less on mechanism, and we have now entered another inflection point. The fifth generation is not only being accelerated by the widespread use of generative AI but is also coinciding with industry-led adoption. This represents a new epoch for how developers fundamentally approach their craft.

As with previous generations maturing, the toolchain to support it has also emerged. In previous generations, tools emerged after a period of stability; however, with the rapid advances in AI research and output, the toolchain now has the potential to shape and define this generation rather than merely support it.

As part of these advancements, a key set of tools has emerged focused on Spec-Driven Development (SDD). This trend is enabled by the acceptance of AI-assisted code generation, which allows developers to elevate their own abstraction and express what a system should do, while intelligent tooling materializes how it is actually done.

This shift redefines how we approach the architecture and design of our systems. Now, instead of static architecture diagrams that drift over time from the original intent, teams maintain living specifications. These define system contracts, boundaries, invariants, and behaviors. Such specifications are executable by design; they can generate code, documentation, SDKs, mocks, and even service infrastructure. AI agents, with the ability to seed their context via persona mapping, in which the expertise of a role is captured in consumable inputs for the agent, can now act authoritatively as interpreters, validators, and domain-specific collaborators throughout this pipeline.

In this article, we examine SDD as an architectural paradigm, detailing how specifications become the executable backbone of a system, how drift detection and continuous validation turn architecture into a runtime invariant, how AI and agentic tooling reshape generation and governance, and how this model represents the next major inflection point in the long evolution of software abstraction.

The Architecture of SDD

The name Spec Driven Development may suggest a methodology, akin to Test

Driven Development. However, this framing undersells its significance. SDD is more accurately understood as an architectural pattern, one that inverts the traditional source of truth by elevating executable specifications above code itself.

SDD represents a fundamental shift in how software systems are architected, governed, and evolved. At a technical level, it introduces a declarative, contract-centric control plane that repositions the specification as the system's primary executable artifact. Implementation code, in contrast, becomes a secondary, generated representation of architectural intent.

Traditional architectures rely on source code as the canonical control surface. In SDD, that control surface moves upward into a specification control plane. This control plane formally allows us to define artifacts such as:

- Interface contracts (capabilities, inputs/outputs, behavioural guarantees)
- Data schemas and invariants (structure, constraints, validation rules)
- Event topologies (allowed flows, sequencing, propagation semantics)
- Security boundaries (identity, trust zones, policy enforcements)
- Compatibility rules (both backwards and forwards)
- Versioning semantics (evolution, depreciation, migration)
- Resource and performance constraints (latency, throughput, costs)

This change encompasses a combination of classical architectural surface areas across behavioral and governance, with a temporal dimension of correctness. Rather than coordinating these areas independently across services and repositories, SDD centralises them into a single authoritative model. This pattern is more closely aligned with a language type system or compiler: it does not execute the program itself but instead defines what is expressible, rejects what is invalid, and constrains evolution to preserve correctness and compatibility over time. Architecture is no longer advisory; it now becomes enforceable and executable.

Despite the growing visibility of tools branded under SDD, it is fundamentally not a product, framework, or formal language. Instead, it is an architectural construct that reappears, with remarkable consistency, as a five-layer execution model.

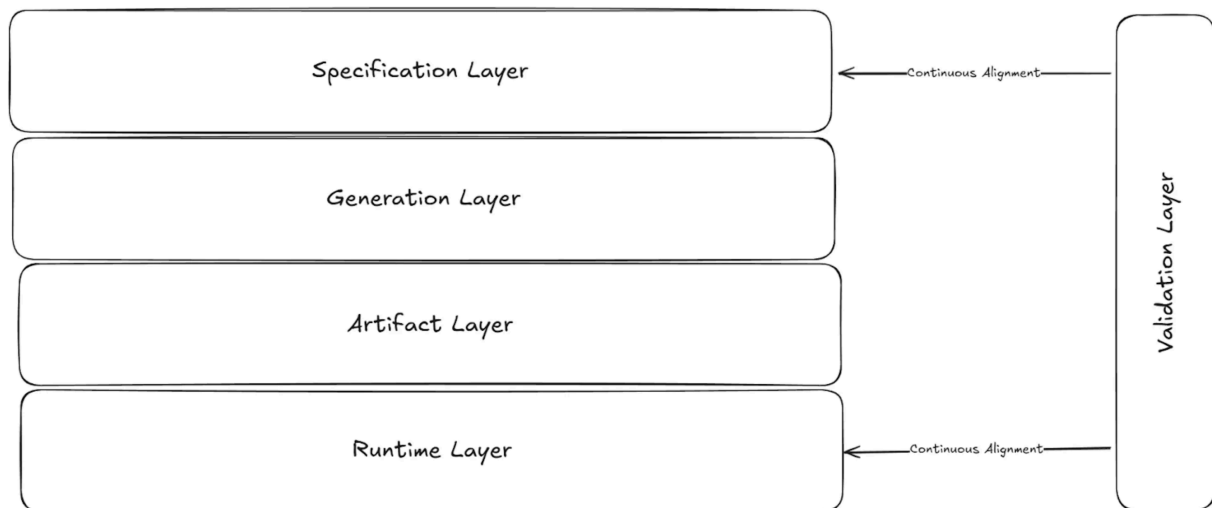


Figure 2: SDD 5 Layer Execution Model

Together, these layers form a closed, specification-governed control system in which intent continuously shapes execution, and execution continuously validates intent. What emerges from this is not incremental improvement to existing architecture, but a fundamental inversion of where authority, control, and truth reside.

Let us examine the five layers now. Throughout, we will follow a simplified example of a classic order management service to show both the progression through the layers and how they reinforce one another.

Specification Layer

This is the authoritative definition of system behavior. It captures the declarative intent of what the system is, not how it is implemented. This layer typically contains API models, messaging contracts, domain schemas, and system-specific, policy-centric constraints. From an abstraction perspective, it is both human-readable and machine-executable, serving simultaneously as a design artifact and an operational control surface.

Taking our order management service, in the Specification Layer, we declare what must be true, not how it is implemented. This is what a pseudo spec of a simple order may look like:

```

service: Orders
api:
  POST /orders:
  
```

```
request:
  Order:
    id: uuid
    quantity: int > 0
responses:
  201: OrderAccepted
  400: ValidationError
```

```
policies:
  compatibility: backward-only
  security:
    auth: mTLS
```

This specification outright states our expectations:

- An order must have a positive quantity
- The API must not introduce breaking changes
- Requests must be authenticated

No language, framework, or infrastructure is referenced here.

Generation Layer

This layer transforms declarative system intent into executable form. It functions as a multi-target system compiler, but unlike classical compilers that emit machine instructions, this layer emits system shapes and enforceable runtime surfaces across languages, frameworks, and platforms. Here, the problem space is declared by the Specification Layer, and the tooling materializes its operational form. Typical outputs include type models, contract stubs, validation middleware, documentation, and a spectrum of integration and conformance tests.

Taking our Order example, the specification is ingested and emits enforceable system surfaces. Conceptually, this would look like:

```
spec.yaml
→ Type models (Java, TypeScript)
→ Request validators
```

- API stubs
- Contract tests

The tooling translates declared intent into concrete, enforceable forms.

Artifact Layer

This layer contains the concrete outputs of the generation phase: generated services, components, clients, data models, and adapters. Critically, these artifacts are not treated as primary assets. Instead, they are regenerable, disposable, replaceable, and continuously reconcilable. This inverts a foundational assumption of traditional software architecture: code is no longer the system of record; the specification is. As code becomes infinitely reproducible and generated on demand, the emerging term Ambient Code aptly captures this paradigm shift.

The shape of our order is now ready to materialise with generated, disposable code. This could see an output like a typed model:

```
export interface Order {  
  id: string;  
  quantity: number;  
}
```

With a validator:

```
if (order.quantity <= 0) {  
  throw new ValidationError("quantity must be greater than 0")  
}
```

These artifacts are not the source of truth. If the specification changes, they are regenerated. If they are deleted, nothing is lost.

Validation Layer

This layer enforces continuous alignment between intent and execution. It consists of contract tests, schema validation, payload inspection, backward compatibility analysis, and architectural drift detection mechanisms. It plays the same structural role that type systems play for programming languages and hypervisors play for virtual machines: actively preventing architectural violations from propagating into the runtime.

The artifacts created by our Generation layer are ultimately governed here, where validation ensures the runtime cannot drift from intent:

✓ `Reject requests with quantity <= 0`

Violations are detected at build time, during deployment, and in our CI systems. Architectural correctness is enforced continuously rather than reviewed manually.

Runtime Layer

This is the operational system itself, composed of a typical mix of artifacts such as :

- APIs
- Message brokers and streaming pipelines
- Functions, methods, and equivalent constructs
- Integration services

Crucially, the runtime's shape is entirely constrained by the upstream specification and validation layers. As a result, runtime behavior becomes architecturally deterministic rather than emergent.

If we try to use our order service with a negative quantity like this:

```
POST /orders
{
  "id": "123",
```

```
"quantity": -1
}
```

We are returned a 400 `ValidationError`, not because the runtime rejected the request, but because the behavior was declared in the Specification Layer, materialized by the Generation Layer, instantiated by the Artifact Layer, and continuously enforced by the Validation Layer, long before the system executed any request.

Architectural Inversion

For decades, software architecture has operated under a largely unchallenged assumption that code is the ultimate authority. Architecture diagrams, design documents, interface contracts, and requirement specifications all existed to guide implementation. However, the running system always derived its truth from what was ultimately deployed. When mismatches occurred, the standard response was to *"update the documentation"*

SDD inverts this relationship entirely. The specification becomes the authoritative definition of system reality, and implementations are continuously derived, validated, and, when necessary, regenerated to conform to that truth. This is not a philosophical distinction; it is a structural inversion of the governance of software systems.

Traditional software delivery follows a linear, lossy pipeline as seen in Figure 3.

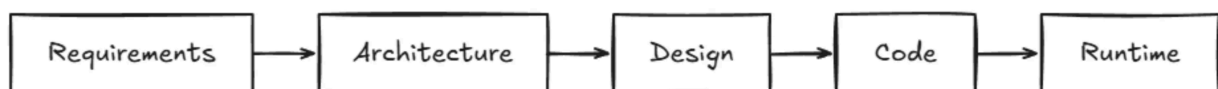


Figure 3: Traditional Software Delivery Pipeline

Each translation step introduces reinterpretation, manual adaptation, and hidden assumptions. Architectural drift is therefore not prevented; it is discovered late, typically through production incidents, failed integrations, security audits, or compliance breaches. By the time misalignment is detected, it is forensic rather than corrective.

SDD fundamentally restructures this flow into a governed control loop:

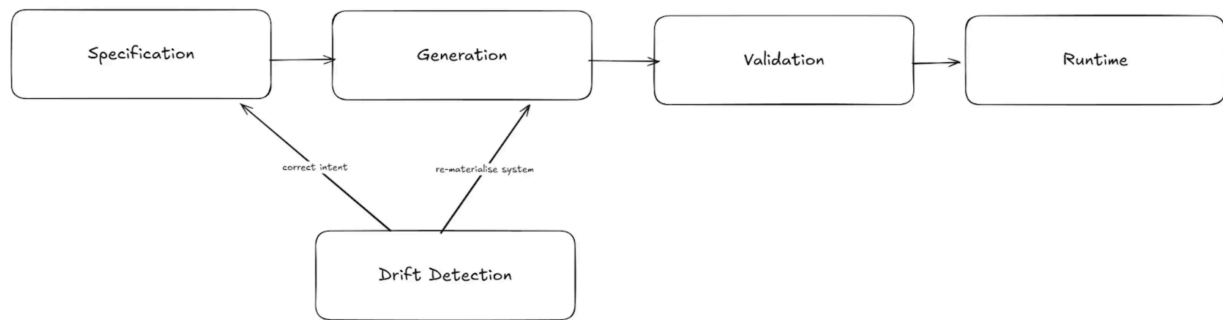


Figure 4: SDD Governed Software Delivery Pipeline

This control loop replaces delayed discovery with active architectural enforcement. Drift detection does not patch runtime behavior; it corrects specification authority and triggers controlled regeneration of the system. Traditional architecture assumes code becomes truth over time; SDD inverts this by ensuring that specification remains the permanent source of truth, and runtime is continuously forced to conform to it.

This architectural inversion can be summarized as follows:

Classical Model	SDD Model
Code defines behavior	Specification defines behavior
Architecture is advisory	Architecture is executable and enforceable
Drift is discovered post fact (or often ignored completely)	Drift is prevented pre-runtime and continuously monitored
Implementation defines truth	Specification defines truth
Validation is retrospective	Validation is continuous and systemic
Runtime is emergent	Runtime is architecturally deterministic

In SDD, code stops being the place where truth emerges and becomes the place where truth is merely realized.

This inversion is structurally equivalent to earlier paradigm shifts where entire classes of correctness constraints were removed from human responsibility and made mechanically enforceable:

- From manual memory management to garbage collection, where memory safety became a runtime invariant
- From bare metal to virtual machines, where isolation and resource boundaries became platform guarantees
- From physical servers to declarative infrastructure, where configuration drift and topology correctness are continuously reconciled
- From untyped languages to statically typed systems, where structural correctness is enforced at compile time
- From informal interface agreements to schema and contract-enforced APIs, where interaction correctness is mechanically validated

In each case, correctness moved from being conventionally enforced by humans to being structurally enforced by the platform. SDD applies this same principle to system boundaries, architecture, and behavior itself.

Drift Detection: Making Architecture Self-Enforcing

Once specifications become authoritative, drift detection is no longer a testing convenience; it becomes a mandatory architectural capability. It is the enforcement mechanism that turns intent into an invariant. In this model, drift is not merely a schema mismatch; it is any divergence between declared system intent and observed system behavior. That divergence may be structural, behavioral, semantic, security-related, or evolutionary. Some examples that we have experienced in our experiments include:

- An API returning fields not declared in the specification
- A service silently omits required fields during refactors
- Message payloads evolving without coordinated schema versioning
- Error handling deviating from contractual guarantees
- Security scopes are degrading relative to their original policy intent

Without drift detection, SDD collapses back into documentation-driven development. With it, the system becomes self policing. Drift detection forms a closed-loop feedback control system. It continuously compares what the system claims to do with what it actually does. This is a fundamentally different operational posture from classical testing, which offers only periodic, sample-based assurance.

In traditional architectures, deviations from intent propagate silently, often for months, until they surface as outages, audit failures, or security breaches. In SDD systems, drift becomes machine detectable by default. Specification validators can be embedded directly into our CI pipelines, and runtime enforcement layers: schema validation, payload inspection, contract verification, and spec differential engines all become first-class architectural components. When outputs violate the specification, the system fails fast and allows for course correction.

This enforcement requirement becomes even more critical in an inherently multi-model future. Software systems will increasingly be shaped by both human-driven development and machine-driven generation, often operating concurrently across the same specification surface. There is no longer a single linear path through the system. Changes may originate from developers, AI agents, automated refactoring tools, or policy-driven generators. This multiplicity of evolutionary paths dramatically amplifies the drift problem: divergence is no longer an edge case; it is the natural state that must be continuously governed.

The net effect is a profound shift in governance. Architecture is no longer a design phase artifact; it becomes a continuously enforced runtime invariant. Specifications shift from passive reference material to active control surfaces, with drift detection serving as the feedback signal that keeps the system aligned with the intent.

However, this does not imply a fully autonomous system in which machines unilaterally define correctness. A spec is not merely a mechanical contract; it is a human expression of purpose, risk tolerance, and trade-off. Drift detection can identify that a system has diverged, but it cannot, on its own, decide whether that divergence is acceptable, accidental, or desirable. Some drift represents defects, while other drift represents evolution. At this boundary, where automated enforcement meets interpretive judgment, the role of the human becomes critical again. Not as a passive reviewer of logs after failure, but as an active participant in

governing meaning, intent, and controlled change. This is where Human-in-the-Loop becomes not a safety net but a first-class design principle.

Human-in-the-Loop: Preserving Intent in an Automated Architecture

When we first explored this mode of system design, we approached it with a naïve "vibe coding" mindset of accepting generated changes with minimal resistance and trusting the SDD toolchain to handle edge cases for us. That assumption failed quickly. What emerged instead was a more powerful realization: SDD does not remove humans from the loop; it relocates human judgment to a higher control plane. The question is no longer how humans implement systems, but how and where they govern them.

SDD does not eliminate human involvement in software design. It reassigns where human cognition is applied. Traditionally, once functionality is implemented, developers have expended the bulk of their effort resolving mismatches, debugging integration failures, reconciling diverged services, and repairing unintended side effects of change. Over time, this became mistakenly synonymous with the craft of software engineering itself. In reality, it is the burden of maintaining large, long-lived, production-facing systems. SDD shifts this burden onto machines, while deliberately retaining human authority over intent, policy, and meaning.

This introduces a new kind of human control surface. Humans remain the ultimate custodians of domain semantics, risk tolerance, safety envelopes, and the evolutionary direction of the system. This authority also extends into the legal, ethical, and moral frameworks that implicitly shape engineering decisions. These dimensions cannot be inferred from execution traces or behavioral observation alone. They exist at a level of abstraction that machines cannot fully own.

Instead, humans explicitly encode these constraints into the Specification Layer and machines assume responsibility for enforcement, generation, and continuous conformance. This mirrors the historical evolution of our craft: just as we once relinquished manual memory management to garbage collection, we are now delegating structural enforcement and mechanical consistency to SDD. What replaces that delegation is not blind automation, but explicit approval boundaries:

- Breaking schema changes require human approval
- Policy shifts require human authorization
- AI proposed refactors require human confirmation
- Compatibility downgrades require human justification

SDD therefore enables bounded autonomy, not full automation, and within these bounds, long-term architectural intent is preserved.

By enforcing drift detection and human oversight of intent, SDD establishes a new division of responsibility between people and machines. Enforcement becomes automated. Meaning remains human. This separation is not philosophical; it is architectural, and precisely this division that gives rise to a new class of foundational system capabilities. A spec native system must now encode enforcement, evolution, validation, and governance directly into its core primitives. We next explore those capabilities and why they are structurally distinct from those in classical software architecture.

Core Capabilities of a Spec Native System

SDD is not enabled by a single tool, framework, or platform. It emerges from a set of tightly coupled architectural capabilities that collectively allow specifications to become executable, enforceable, and evolvable at scale. When any of these capabilities is absent, SDD reverts to documentation-driven development or ad hoc code generation. To move from theory into an operational paradigm, a system must internalize five core capabilities:

1. Spec authoring as a first-class engineering surface
2. Formal validation and type enforcement
3. Deterministic generation and composition
4. Continuous conformance and drift enforcement
5. Governed evolution and compatibility control

We refer to this operational discipline as SpecOps, short for Specification Operations. From a SpecOps perspective, in which specifications are treated as first-class, executable system assets, these capabilities do not define a product category; they represent a control plane for software intent. In a spec-native system, specification authoring is not an activity that precedes implementation; it is the implementation activity. The system must therefore support multi-model specifications in which structural, behavioral, and policy definitions coexist within

a unified schema space. This requires composable domain modeling, so that layered specifications become a viable architectural strategy rather than a documentation convenience.

With specifications becoming the primary system artifact, they must be treated with the same operational rigor as source code: version control, peer review, branching, and controlled merge strategies all become mandatory. This is the moment at which the specification ceases to be descriptive and becomes a programmable model of the system itself.

Once a specification is executable, it must also be machine verifiable with the same rigor as a compiler frontend or a type system. This enforcement spans structural validation, semantic coherence, and domain-invariant enforcement. Conditional constraints, referential integrity, and cross-spec consistency must be provable. The effect is not merely improved correctness, it is the elimination of entire classes of system failure from the space of what can be represented at all, in precisely the way static typing constrains illegal programs.

Generation, in this paradigm, is not a form of scaffolding. It is the materialization of declared system truth. This requires strictly deterministic behavior. A production-grade spec native system must guarantee input determinism: identical specifications must always yield identical artifacts. It must be target agnostic, producing consistent outputs across languages, platforms, and runtime environments. Most critically, generation must be logically reversible. The system must always be able to answer a simple but foundational question: Which specification state produced this behavior? This traceability of the lineage of a decision is what elevates generation from productivity aid to architectural authority.

Once generation is automated, enforcement necessarily becomes continuous. Runtime systems can no longer diverge quietly from declared intent. Implementations cannot introduce undocumented behavior. Consumers cannot rely on undefined properties. Architecture, therefore, transitions from a design-time assertion into a runtime invariant, actively maintained by the system itself.

The most difficult capability in SDD is not generation or validation, but change without fracture. A spec native system must automatically classify changes as additive, compatible, breaking, or ambiguous, and enforce an explicit compatibility policy. This introduces a formal notion of governed evolution:

parallel version surfaces, known compatibility windows, controlled deprecation curves, and explicit approval gates for breaking change are required. Without this, SDD becomes architecturally brittle. With it, the system can evolve without violating its own correctness guarantees.

The most profound shift introduced by these five capabilities is not technical; it is structural.

The unit of delivery is no longer a service or a codebase. The unit of delivery becomes the specification itself. This realigns outcomes with outputs: what is declared is what is delivered. This stands in direct contrast to vibe-driven, generative coding approaches, in which deviation is an emergent property of creativity (or hallucination), rather than a governed act of design.

Conclusion: Engineering Tradeoffs and Challenges Exist

Every major abstraction leap in software engineering has delivered extraordinary productivity gains, while simultaneously introducing entire new classes of systemic risk. Garbage collection eliminated vast swathes of memory errors while introducing pause time behavior and new failure modes. Virtual machines simplified deployment while creating orchestration complexity. Cloud platforms removed infrastructure burden while introducing deep operational coupling. Spec Driven Development is no exception.

By elevating the system's source of truth into specifications and generators, SDD does not remove complexity; it simply relocates it. The trade-offs below define the real engineering costs we have experienced while adopting this paradigm at scale.

Specifications Become a Primary Complexity Surface

In SDD, specifications cease to be documentation artifacts and become long-lived executable infrastructure. As a result, they acquire properties traditionally associated with source code. They inherit all the properties usually associated with source code: technical debt, cross-team coupling, compatibility inertia, and architectural gravity. Schema engineering, therefore, becomes a first-class architectural discipline, on par with data modeling and distributed systems design.

Generator Trust Becomes a Supply Chain Problem

In SDD, AI code generators are no longer developer conveniences. They become structural components of the system's trusted computing base. Determinism, reproducibility, auditability, sandboxed execution, and verifiable provenance are no longer optional properties; they are mandatory. Code generation is elevated from tooling to critical infrastructure.

Runtime Enforcement Has a Real Cost

SDD shifts enforcement from social process to technical control. That shift is powerful, but it is not free. Runtime contract validation introduces real computational overhead. At a small scale, this cost is negligible. At a large scale, we need to contend with the system's purpose, be it high-frequency APIs, real-time streams, or latency-sensitive systems. This becomes an explicit architectural budget item. Correctness becomes a metered resource rather than a default-free property.

The Cognitive Shift is Non-Trivial

SDD replaces implementation-first thinking with contract-first reasoning. This requires engineers to adopt new mental models:

- Thinking in invariants instead of behaviors
- Reasoning about compatibility rather than features
- Expressing intent declaratively instead of procedurally
- Treating schemas as executable programs

Every historical abstraction shift expanded human leverage while introducing unfamiliar failure modes that took years to master. SDD is now entering the same maturation curve.

The Price of Architectural Authority

While new paradigm shifts are often exciting, the decision to adopt this ultimately comes down to balancing the practical trade-offs involved.

On one hand, SDD delivers:

- architectural determinism
- continuous correctness enforcement
- systemic reduction of drift
- Multi-language parity
- reproducible system boundaries

But it exacts its price in:

- schema complexity
- generator trust requirements
- runtime validation cost
- Long-term compatibility burden
- cognitive transformation of engineering roles

This is not a reason to avoid SDD. It is a reason to adopt it deliberately, with explicit governance, disciplined specification practice, and sober recognition of its costs. Every leap in abstraction demands new forms of rigor.

SDD simply relocates that rigor to where it has always belonged: the definition of system truth itself.

About the Authors



Leigh Griffin

Leigh holds a PhD in Computer Science and has a passion for coaching Agile teams for continuous improvement. Leigh works for the Office of the CTO at Red Hat helping to explore emerging technologies and the intersection of them with the process and flow of work into Engineering teams.

Show more

Show less



Ray Carroll

Ray Carroll is a Senior Principal Engineer in Red Hats Emerging Technology group, focused on investigating, evaluating and developing new and emerging technologies. Ray holds a Phd in Computer Science, with a background in both academic research and various industry roles such as Software Engineer, Research Architect and Technical Architect.

Show more

Show less

Please see <https://www.infoq.com> for the latest version of this information.